

Tarea MPI

Jorge Grez

June 2026

Respuesta ítem a

El problema nos plantea la ecuación de Poisson con un término no nulo $f(p)$:

$$\Delta\phi(p) = f(p)$$

En un espacio bidimensional, el operador laplaciano $\Delta\phi$ se expande como :

$$\frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} = f(x, y)$$

Para implementar esto en una matriz bidimensional, discretizamos la ecuación utilizando el método de diferencias finitas centradas. Bajo esto las segundas derivadas se aproximan de la siguiente manera:

$$\begin{aligned}\frac{\partial^2\phi}{\partial x^2}(x_i) &\approx \phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j} \\ \frac{\partial^2\phi}{\partial y^2}(y_i) &\approx \phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}\end{aligned}$$

Donde por notación tenemos $\phi_{i,j} = \phi(i, j)$ considerando la distancia entre puntos adyacentes $h = 1$.

Sustituyendo estas aproximaciones espaciales en nuestra ecuación original, obtenemos:

$$(\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}) + (\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}) = f_{i,j}$$

Agrupando los términos correspondientes al punto central $\phi_{i,j}$, la ecuación queda como:

$$\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - 4\phi_{i,j} = f_{i,j}$$

Para aplicar el método de iteración, se necesita una regla de actualización que permita calcular el nuevo valor del punto central basándonos en sus vecinos. Por lo tanto, despejando analíticamente $\phi_{i,j}$ se tiene que:

$$4\phi_{i,j} = \phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - f_{i,j}$$

$$\phi_{i,j} = \frac{\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - f_{i,j}}{4}$$

Adaptando esto a la implementación computacional. La función ϕ corresponde a los datos bidimensionales de la matriz `data`. El término heterogéneo f corresponde a su versión discretizada ρ , la cual mantiene la misma dimensionalidad que la matriz de datos. Reemplazando estos términos, se obtiene la actualización de `data` buscada:

$$data[i,j] \leftarrow \frac{data[i+1,j] + data[i-1,j] + data[i,j+1] + data[i,j-1] - rho[i,j]}{4}$$

Respuesta ítem b (Ver código adjunto)

La estrategia computacional para la implementación del algoritmo consistió en los siguientes puntos clave:

- **Descomposición del Dominio (Topología Cartesiana 2D):** La matriz global que representa la cuadrícula espacial no se dividió en franjas unidimensionales, sino que se particionó en bloques bidimensionales. Utilizando la función `MPI.Cart_create`, se estructuró un comunicador con topología cartesiana, permitiendo que cada proceso identificara automáticamente a sus cuatro vecinos cardinales (Norte, Sur, Este y Oeste).
- **Gestión de Celdas Fantasma (Halos):** Para calcular la regla de actualización en los bordes de cada bloque local, se expandió la memoria de cada submatriz añadiendo celdas fantasma (*halos*). Estas celdas almacenan temporalmente el estado de las fronteras pertenecientes a los procesos vecinos.
- **Tipos de Datos Derivados para Columnas:** Dado que en el lenguaje C las matrices se almacenan en memoria con un formato de fila principal (*row-major*), las filas son bloques de memoria contiguos que se pueden enviar directamente. Sin embargo, los elementos de una columna están fragmentados con saltos en la memoria. Para resolver esto sin penalizar el rendimiento copiando elementos uno por uno a un búfer temporal, se definió un tipo de dato derivado mediante `MPI.Type_vector`. Esto instruyó a MPI sobre cómo recolectar y transmitir columnas enteras (fronteras Este y Oeste) de manera eficiente y directa.
- **Comunicación no bloqueante:** El intercambio de los halos se realizó mediante operaciones asíncronas (`MPI_Isend` y `MPI_Irecv`) seguidas de una barrera de espera (`MPI_Waitall`), lo que permitió ejecutar múltiples transferencias de red de forma simultánea antes de proceder con el cálculo iterativo de Jacobi. Se aplicaron condiciones de frontera de Dirichlet (valor cero) en los límites absolutos del dominio global.

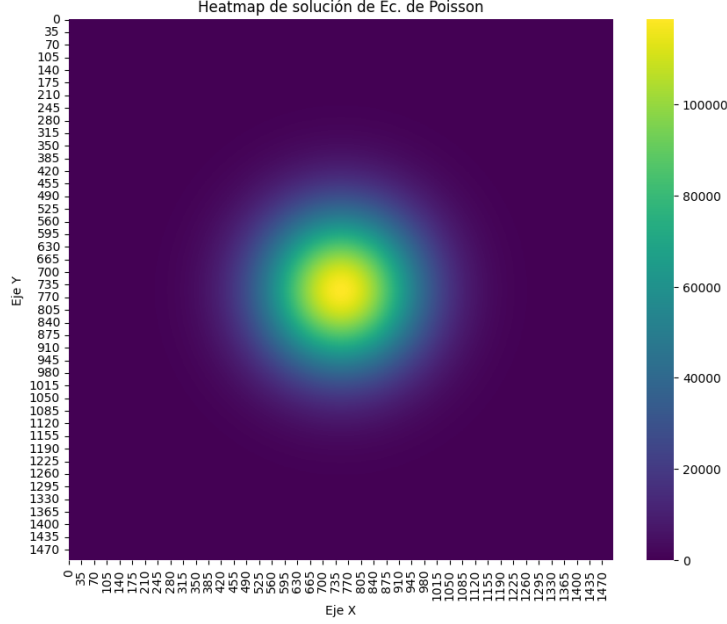


Figure 1: Heatmap con resultados

El término heterogéneo $f(x, y)$, representado en el algoritmo por la matriz de densidad **rho**, fue modelado matemáticamente utilizando una **distribución Gaussiana bidimensional**.

La expresión analítica implementada es la siguiente:

$$f(x, y) = A \cdot \exp\left(-\frac{(x - x_c)^2 + (y - y_c)^2}{2\sigma^2}\right) \quad (1)$$

Respuesta ítem c

Se corrieron los programas con topología 1D y 2D en Kosmos, ambos para una malla de 1500×1500 para 1, 2, 4, 8, 16 y 32 procesos 4 veces por cada uno para obtener un promedio por cada cantidad de procesos. Los resultados experimentales obtenidos (figura 3) indican que la partición unidimensional presenta tiempos de ejecución equivalentes, e incluso levemente inferiores, a la partición bidimensional (2D). Este comportamiento se fundamenta en la interacción de los siguientes factores algorítmicos:

- **Costo de Latencia vs. Ancho de Banda:** El tiempo total de comunicación en MPI depende tanto del tamaño del mensaje (ancho de banda) como del número de conexiones iniciadas (latencia). Para este tamaño de malla, los paquetes de datos no son lo suficientemente grandes para

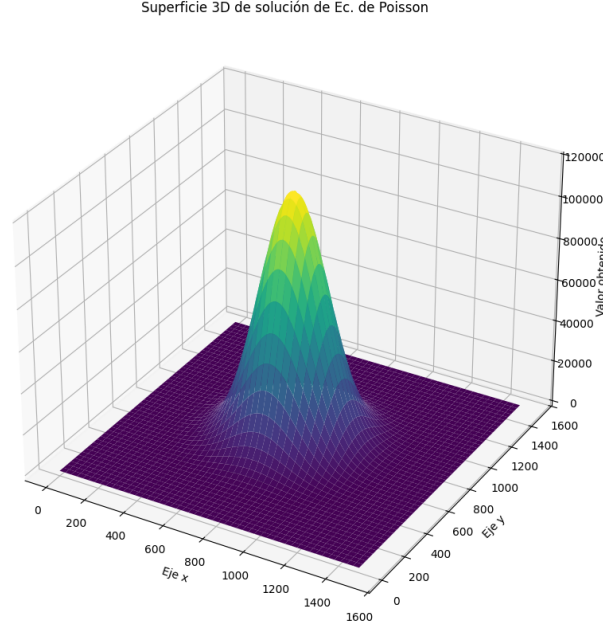


Figure 2: Superficie 3D con resultados

saturar la red. En consecuencia, la latencia se convierte en el factor dominante. La topología 1D resulta más rápida al requerir solo 2 transferencias por iteración (Norte y Sur), frente a las 4 requeridas por la topología 2D (Norte, Sur, Este y Oeste).

- **Patrón de Acceso a Memoria (Caché):** En el lenguaje C, los arreglos se almacenan secuencialmente por filas (*row-major order*). La partición 1D se beneficia de este diseño al enviar y recibir filas completas (memoria contigua), maximizando la tasa de aciertos (*hit rate*) en la memoria caché del procesador. Por el contrario, la partición 2D debe extraer datos no contiguos para las fronteras verticales (Este y Oeste) mediante `MPI_Type_vector`, introduciendo ciclos de CPU adicionales para el empaquetado y desempaquetado de datos.
- **Ocultamiento de Comunicación (*Communication Hiding*):** Con 2.25 millones de celdas en el dominio, el esfuerzo computacional para actualizar los valores internos de la matriz ($\mathcal{O}(N^2)$) es significativamente mayor que el tiempo empleado en actualizar los halos de los bordes ($\mathcal{O}(N)$). Este denso trabajo aritmético superpone y oculta las ventajas teóricas de comunicación esperadas en el modelo 2D.

Conclusión: A esta escala, la partición 1D es más eficiente al evitar las penalizaciones de memoria fragmentada y minimizar las llamadas a la red (laten-

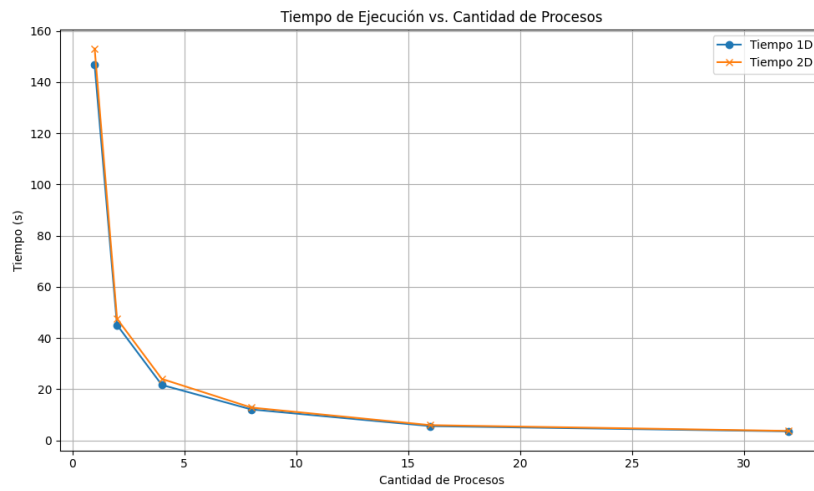


Figure 3: Tiempos de computo

cia). La asintótica superioridad de la topología 2D requiere estar en escenarios de mayor estrés: mallas globales masivas ejecutadas a través de múltiples nodos físicos independientes.